1. **What are the most common reasons for missing data in ETL pipelines?**
   Missing data in ETL pipelines is very common, and it usually happens due to issues at different stages—source, extraction, transformation, or loading. Here are the most common reasons:
   1. Data Not Collected at Source
   Sometimes the data simply isn't captured in the first place.
   Example: A user skips filling the "Age" field in a form.
   2. System or Sensor Failures
   Hardware or software systems can fail while collecting data.
   Example: IoT sensors going offline or APIs not responding.
   3. Data Entry Errors
   Manual input often leads to missing or incomplete fields.
   Example: Typing mistakes or leaving fields blank.
   4. Data Integration Issues
   When combining multiple data sources, some fields may not match or exist across all sources.
   Example: One system has "Phone Number" while another doesn't.
   5. Schema Mismatch or Changes
   Changes in database structure can cause missing fields during ETL.
   Example: A column renamed or removed in the source system.
   6. Filtering During Transformation
   Data might be intentionally removed during cleaning or transformation.
   Example: Dropping rows with invalid or null values.
   7. Extraction Failures
   Incomplete extraction due to network issues, timeouts, or misconfigured queries.
   Example: API returns partial data due to rate limits.
   8. Data Corruption
   Files or records may get corrupted during transfer or storage.
   Example: Broken CSV file leading to missing values.
   9. Permission or Access Issues
   ETL jobs may not have access to all required data.
   Example: Restricted columns not accessible due to security rules.
   10. Timing Issues (Latency)
   Data may not be available at the time of extraction.
   Example: Late-arriving data in streaming pipelines.
2. **Why is blindly deleting rows with missing values considered a bad practice in ETL?**
   Blindly deleting rows with missing values might seem like a quick fix, but it's usually a poor practice in ETL because it can seriously damage the quality and usefulness of your data.
   1. Loss of Valuable Information
   You may throw away useful data just because one field is missing.
   Example: A row missing "Age" might still have important "Sales" or "Customer ID" data.
   2. Reduced Dataset Size

Deleting many rows can shrink your dataset significantly, which affects analysis quality and model performance.

3. Introduces Bias

Missing data is often not random. Removing those rows can distort the dataset.
Example: If high-income users tend to skip certain fields, deleting them biases your results toward lower-income users.

4. Breaks Data Representativeness

Your dataset may no longer reflect the real-world population after deletion.

5. Impacts Downstream Analysis / ML Models

Machine learning models rely on sufficient and balanced data. Less data = weaker, less accurate models.

6. Can Remove Rare but Important Cases

Outliers or rare events (like fraud cases) often have incomplete data. Deleting them can remove critical insights.

7. Ignores Root Cause

Blind deletion doesn't solve why the data is missing (system error, pipeline issue, etc.). The problem may continue.

3. **Explain the difference between: Listwise deletion Column deletion**

1. Listwise Deletion (Row Deletion)

- Remove entire rows if any value is missing.

When it is appropriate:
Use it when missing data is very small and random (doesn't bias the dataset).
Scenario:
You have a dataset of 10,000 customer records, and only 50 rows have missing values in any column.
Deleting those 50 rows won't impact overall analysis.
Why it works here:

- Data loss is minimal
- Missingness is likely random
- Dataset remains representative

2. Column Deletion (Feature Deletion)

- Remove entire columns with too many missing values.

When it is appropriate:
Use it when a column has very high missing rate and is not critical.
Scenario:
In a customer dataset, the "Middle Name" column is 90% missing.
It doesn't add much value for analysis or modeling.
Why it works here:

- Column has little usable data
- Not important for business decisions
- Removing it simplifies the dataset

4. **Why is median imputation preferred over mean imputation for skewed data such as income?**

Median imputation is preferred over mean imputation for skewed data (like income) because the median is resistant to extreme values, while the mean is not.

Key Reason: Effect of Outliers

Income data is usually right-skewed (a few people earn very high incomes).

- Mean gets pulled toward those extreme high values
- Median stays near the "middle" of the typical population

Example

Suppose incomes are:

₹20k, ₹25k, ₹30k, ₹35k, ₹5,00,000

- Mean = very high (affected by ₹5,00,000)
- Median = ₹30k (represents typical income better)

Why Median is Better

- Robust to outliers → not affected by extreme values
- Represents typical value → better for skewed distributions
- Reduces bias in the dataset
- Keeps data closer to real-world patterns

Problem with Mean Imputation

- Distorts the distribution
- Overestimates or underestimates missing values
- Can negatively impact analysis and ML models

5. **What is forward fill and in what type of dataset is it most useful?**

   Forward fill is a method of handling missing values where you replace a missing value with the last known (previous) valid value in the dataset.

   When is it most useful?

   Forward fill is best for time-series or sequential data, where:

- Data is ordered (by time, date, or sequence)
- Previous values logically carry forward until updated

Common Use Cases

- Stock prices (price remains same until next trade)
- Sensor/IoT data (last reading assumed until new reading)
- Sales or inventory tracking
- Weather data

6. **Why should flagging missing values be done before imputation in an ETL workflow?**

   Flagging missing values before imputation is important because once you fill the data, you lose information about what was originally missing—and that information can be very valuable.

   Imputation hides missingness.

   Flagging preserves it.

   1. Preserve Important Information

   Missing data itself can carry meaning.

   Example: If "Income" is missing, it might indicate a specific user group.

   A flag column like Income_missing = 1 keeps this signal.

   2. Improve Model Performance

   Machine learning models can use the **missingness pattern** as a feature.

   Sometimes, "missing" is predictive (e.g., people who skip fields behave differently).

   3. Avoid Data Misinterpretation

   After imputation, all values look "complete," and you can't tell which ones were filled artificially.

This can mislead analysis and decision-making.

4. Better Transparency & Debugging

Flags help track:

- How much data was missing
- Where imputation was applied

Useful for auditing and troubleshooting ETL pipelines.

5. Flexibility for Future Improvements

If you later want to try a different imputation method, flags help you easily identify affected records.

7. **Consider a scenario where income is missing for many customers. How can this missingness itself provide business insights?**

1. Privacy sensitivity / trust level

Customers who skip income may be more privacy-conscious.
Insight: They might respond better to **trust-building messaging** and minimal data requests.

2. Segment identification

Missingness can cluster by user type (e.g., students, freelancers, new users).
Insight: Create a segment like **"Income Unknown"** and analyze its behavior separately.

3. Conversion or funnel friction

If many users drop or skip at the income field, it may be a **UX issue**. Insight: That field might be hurting sign-ups or completions—consider making it optional or deferring it.

4. Risk or credit behaviour signal

In finance, not disclosing income can correlate with **higher uncertainty or risk**.
Insight: Use it as an input in **risk scoring** (with caution and fairness checks).

5. Product relevance / targeting gaps

Users may skip income because they don't see why it's needed.
Insight: Your product messaging or value proposition may not be clear for that segment.

6. Channel or geography patterns

Missingness may be higher for certain acquisition channels or regions.
Insight: Some channels bring **lower-intent or less-engaged users**, or certain regions are more privacy-sensitive.

8.

Customer_ID	Name	City	Monthly_Sales	Income	Region
101	Rahul Mehta	Mumbai	12000	65000	West
102	Anjali Rao	Bengaluru	NaN	NaN	South
103	Suresh Iyer	Chennai	15000	72000	South
104	Neha Singh	Delhi	NaN	NaN	North
105	Amit Verma	Pune	18000	58000	NaN
106	Karan Shah	Ahmedabad	NaN	61000	West
107	Pooja Das	Kolkata	14000	NaN	East
108	Riya Kapoor	Jaipur	16000	69000	North

Listwise Deletion: Remove all rows where Region is missing.

Tasks: Identify affected rows .Show the dataset after deletion .Mention how many records were lost.

- Customer_ID 105 (Amit Verma) → Region = NaN

So, only 1 row is affected.

Dataset after deletion:

Customer_ID	**Name**	**City**	**Monthly_Sales**	**Income**	**Region**
101	Rahul Mehta	Mumbai	12000	65000	West
102	Anjali Rao	Bengaluru	NaN	NaN	South
103	Suresh Iyer	Chennai	15000	72000	South
104	Neha Singh	Delhi	NaN	NaN	North
106	Karan Shah	Ahmedabad	NaN	61000	West

Customer_ID	Name	City	Monthly_Sales	Income	Region
107	Pooja Das	Kolkata	14000	NaN	East
108	Riya Kapoor	Jaipur	16000	69000	North

Number of records lost

- Original records = **8**
- Remaining records = **7**

Records lost = 1

9. Imputation: Handle missing values in Monthly_Sales using: Forward Fill Tasks: Apply forward fill. Show before vs after values. Explain why forward fill is suitable here

Rows with missing values:

- Customer_ID 102 (Anjali Rao)
- Customer_ID 104 (Neha Singh)
- Customer_ID 106 (Karan Shah)

After applying forward fill:

Customer_ID	Name	Monthly_Sales (Before)	Monthly_Sales (After fill)
101	Rahul Mehta	12000	12000
102	Anjali Rao	NaN	12000
103	Suresh Iyer	15000	15000
104	Neha Singh	NaN	15000
105	Amit Verma	18000	18000
106	Karan Shah	NaN	18000
107	Pooja Das	14000	14000
108	Riya Kapoor	16000	16000

Why Forward Fill is Suitable Here

- The data appears ordered (by Customer_ID / sequence)
- Assumes last known sales value continues until updated
- Useful when values don’t change drastically between records

10. Flagging Missing Data: Create a flag column for missing Income. Tasks: Create Income_Missing_Flag (0 = present, 1 = missing). Show updated dataset. Count how many customers have missing income

Rule:

- 0 → Income present
- 1 → Income missing (NaN)

Customer_ID	Name	City	Monthly_Sales	Income	Region	Income_Missing_Flag
101	Rahul Mehta	Mumbai	12000	65000	West	0
102	Anjali Rao	Bengaluru	NaN	NaN	South	1
103	Suresh Iyer	Chennai	15000	72000	South	0
104	Neha Singh	Delhi	NaN	NaN	North	1
105	Amit Verma	Pune	18000	58000	NaN	0
106	Karan Shah	Ahmedabad	NaN	61000	West	0
107	Pooja Das	Kolkata	14000	NaN	East	1
108	Riya Kapoor	Jaipur	16000	69000	North	0

Count missing Income

Customers with missing income:

- 102 (Anjali Rao)
- 104 (Neha Singh)
- 107 (Pooja Das)

Total missing = 3 customers